

手摸手实现Web组件库

组件地址

```
<dependency>
  <groupId>org.opengoofy.index12306</groupId>
  <artifactId>index12306-web-spring-boot-starter</artifactId>
  <version>${project.version}</version>
</dependency>
```

组件概述

1. 定义全局异常处理器。
2. 全局返回对象构造器。
3. 项目启动后初始化调用 HTTP 请求，增加第一次访问速度。

组件功能

全局异常处理器

定义全局异常处理器，避免请求处理过程中出现代码中未曾捕获的异常返回到前端。

如果对该概念不理解，可以查看详细的文档。

[只会控制层捕获异常？可以尝试全局拦截](#)

```
package org.opengoofy.index12306.framework.starter.web;

import cn.hutool.core.collection.CollectionUtil;
import cn.hutool.core.util.StrUtil;
import jakarta.servlet.http.HttpServletRequest;
import lombok.SneakyThrows;
import lombok.extern.slf4j.Slf4j;
import org.opengoofy.index12306.framework.starter.convention.errorcode.BaseErrorCode;
import org.opengoofy.index12306.framework.starter.convention.exception.AbstractException;
```

```

import org.opengoofy.index12306.framework.starter.convention.result.Result;
import org.springframework.util.StringUtils;
import org.springframework.validation.BindingResult;
import org.springframework.validation.FieldError;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.util.Optional;

/**
 * 全局异常处理器
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Slf4j
@RestControllerAdvice
public class GlobalExceptionHandler {

    /**
     * 拦截参数验证异常
     */
    @SneakyThrows
    @ExceptionHandler(value = MethodArgumentNotValidException.class)
    public Result validExceptionHandler(HttpServletRequest request,
    MethodArgumentNotValidException ex) {
        BindingResult bindingResult = ex.getBindingResult();
        FieldError firstFieldError =
CollectionUtil.getFirst(bindingResult.getFieldErrors());
        String exceptionStr = Optional.ofNullable(firstFieldError)
            .map(FieldError::getDefaultMessage)
            .orElse(StrUtil.EMPTY);
        log.error("[{}] {} [ex] {}", request.getMethod(), getUrl(request), exceptionStr);
        return Results.failure(BaseErrorCode.CLIENT_ERROR.code(), exceptionStr);
    }

    /**
     * 拦截应用内抛出的异常
     */
    @ExceptionHandler(value = {AbstractException.class})
    public Result abstractException(HttpServletRequest request, AbstractException ex) {

```

```

        if (ex.getCause() != null) {
            log.error("[{}] {} [ex] {}", request.getMethod(),
request.getRequestURL().toString(), ex.toString(), ex.getCause());
            return Results.failure(ex);
        }
        log.error("[{}] {} [ex] {}", request.getMethod(),
request.getRequestURL().toString(), ex.toString());
        return Results.failure(ex);
    }

    /**
     * 拦截未捕获异常
     */
    @ExceptionHandler(value = Throwable.class)
    public Result defaultErrorHandler(HttpServletRequest request, Throwable throwable) {
        log.error("[{}] {} ", request.getMethod(), getUrl(request), throwable);
        return Results.failure();
    }

    private String getUrl(HttpServletRequest request) {
        if (StringUtils.isEmpty(request.getQueryString())) {
            return request.getRequestURL().toString();
        }
        return request.getRequestURL().toString() + "?" + request.getQueryString();
    }
}

```

返回对象构造器

配合规约中的 Result 全局返回对象使用，Results 提供 Result 的快捷构建。

```

package org.opengoofy.index12306.framework.starter.web;

import org.opengoofy.index12306.framework.starter.convention.errorcode.BaseErrorCode;
import org.opengoofy.index12306.framework.starter.convention.exception.AbstractException;
import org.opengoofy.index12306.framework.starter.convention.result.Result;

import java.util.Optional;

/**

```

```
* 全局返回对象构造器
*
* @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
*/
public final class Results {

    /**
     * 构造成功响应
     */
    public static Result<Void> success() {
        return new Result<Void>()
            .setCode(Result.SUCCESS_CODE);
    }

    /**
     * 构造带返回数据的成功响应
     */
    public static <T> Result<T> success(T data) {
        return new Result<T>()
            .setCode(Result.SUCCESS_CODE)
            .setData(data);
    }

    /**
     * 构建服务端失败响应
     */
    protected static Result<Void> failure() {
        return new Result<Void>()
            .setCode(BaseErrorCode.SERVICE_ERROR.code())
            .setMessage(BaseErrorCode.SERVICE_ERROR.message());
    }

    /**
     * 通过 {@link AbstractException} 构建失败响应
     */
    protected static Result<Void> failure(AbstractException abstractException) {
        String errorCode = Optional.ofNullable(abstractException.getErrorCode())
            .orElse(BaseErrorCode.SERVICE_ERROR.code());
        String errorMessage = Optional.ofNullable(abstractException.getErrorMessage())
            .orElse(BaseErrorCode.SERVICE_ERROR.message());
        return new Result<Void>()

```

```

        .setCode(errorCode)
        .setMessage(errorMessage);
    }

    /**
     * 通过 errorCode、errorMessage 构建失败响应
     */
    protected static Result<Void> failure(String errorCode, String errorMessage) {
        return new Result<Void>()
            .setCode(errorCode)
            .setMessage(errorMessage);
    }
}

```

优化项目初次调用响应

不知道大家有没有注意过，SpringBoot 第一次调用的接口影响速度，相比对之后的，显得格外的慢。

为此，我在 Web 中定义了伪 Controller 接口，并在项目启动后进行自己调用自己。极大优化了第一次调用的接口性能响应。

1) 定义伪 Controller 接口，方便自己调用自己。

```

package org.opengoofy.index12306.framework.starter.web.initialize;

import lombok.extern.slf4j.Slf4j;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import static
    org.opengoofy.index12306.framework.starter.web.config.WebAutoConfiguration.INITIALIZE_PATH;

/**
 * 初始化 {@link org.springframework.web.servlet.DispatcherServlet}
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@Slf4j(topic = "Initialize DispatcherServlet")

```

```

@RestController
public final class InitializeDispatcherServletController {

    @GetMapping(INITIALIZE_PATH)
    public void initializeDispatcherServlet() {
        log.info("Initialized the dispatcherServlet to improve the first response time of
the interface...");
    }
}

```

2) 继承 `CommandLineRunner` 接口，在项目启动后进行自调用。

```

import lombok.RequiredArgsConstructor;
import org.springframework.boot.CommandLineRunner;
import org.springframework.core.env.ConfigurableEnvironment;
import org.springframework.http.HttpMethod;
import org.springframework.web.client.RestTemplate;
import org.springframework.web.servlet.DispatcherServlet;

import static
org.opengoofy.index12306.framework.starter.web.config.WebAutoConfiguration.INITIALIZE_PAT
H;

/**
 * 通过 {@link InitializeDispatcherServletController} 初始化 {@link DispatcherServlet}
 *
 * @公众号：马丁玩编程，回复：加群，添加马哥微信（备注：12306）获取项目资料
 */
@RequiredArgsConstructor
public final class InitializeDispatcherServletHandler implements CommandLineRunner {

    private final RestTemplate restTemplate;

    private final ConfigurableEnvironment configurableEnvironment;

    @Override
    public void run(String... args) throws Exception {
        String url = String.format("http://127.0.0.1:%s%s",
            configurableEnvironment.getProperty("server.port", "8080") +
configurableEnvironment.getProperty("server.servlet.context-path", ""),
            INITIALIZE_PATH);
    }
}

```

```
    try {  
        restTemplate.execute(url, HttpMethod.GET, null, null);  
    } catch (Throwable ignored) {  
    }  
}  
}
```

有同学提出疑问，为什么不使用 MVC 初始化？

```
spring:  
  mvc:  
    servlet:  
      load-on-startup: 1
```

我这边测试了下，通过正常流程，调用网关服务，网关服务调用聚合中的某个返回（返回一个对象）接口，测试数据如下：

默认请求

- 142ms
- 135ms
- 149ms

MVC 初始化

- 142ms
- 140ms
- 139ms

Web 组件库初始化

- 26ms

更新: 2024-04-14 14:36:53

原文: <<https://www.yuque.com/magestack/12306/rkmeoo92ktipoxln>>